🔥 Complete Beginner Guide: The Two GEE Scripts, Predicted Points, and Visualization

> ***This document explains EVERYTHING from scratch.*** *No prior knowledge assumed. It covers: where data points come from, why they are NOT random, how CSV maps to GEE, and every line of both JavaScript files.*

TABLE OF CONTENTS

1. The Big Picture — What Are We Doing?
2. SCRIPT 1: Data Extraction (`GEE_WesternGhats_AllParameters.js`) — Line by Line
3. The Journey: GEE → CSV → Python → Predicted Points CSV
4. SCRIPT 2: Visualization (`GEE_RF_FirePrediction_Visualization.js`) — Line by Line
5. FAQ: Are the Points Random? How Are They Mapped?

---

1. THE BIG PICTURE — What Are We Doing?

Imagine this pipeline as a 4-step relay race:

```
STEP 1 (GEE Script 1)          STEP 2 (Download)
Satellites → Sample Points  →  CSV files on Google Drive
    in GEE                         with lat/lon + features

STEP 3 (Python Notebook)        STEP 4 (GEE Script 2)
CSV → Train ML Model      →   Take predicted lat/lon
   → Predict on test data       → Plot them back on the map
   → Save results CSV
```

What is Google Earth Engine (GEE)?

GEE is a web platform by Google. It has a **massive archive of satellite images** (NASA, ESA, etc.) going back 40+ years. Instead of downloading terabytes to your laptop, you write JavaScript code that runs **on Google's cloud servers** — they do the processing and give you results.

Website: `code.earthengine.google.com`

What is our goal?

Predict where forest fires will happen in the Western Ghats region of India using satellite-derived environmental features and machine learning.

---

2. SCRIPT 1: Data Extraction — `GEE_WesternGhats_AllParameters.js`

PURPOSE: Extract satellite data for 40,000 locations and save as CSV files.

2.1 Defining the Study Area (Lines 46–56)

```
var ecoregions = ee.FeatureCollection("RESOLVE/ECOREGIONS/2017");
```

What this does: Loads a global dataset of ecological regions. Think of this as a world map where every region is labeled by its ecosystem type (rainforest, desert, grassland, etc.).

```
var westernGhats = ecoregions.filter(ee.Filter.or(
  ee.Filter.eq('ECO_NAME', 'South Western Ghats moist deciduous forests'),
  ee.Filter.eq('ECO_NAME', 'North Western Ghats moist deciduous forests'),
  ee.Filter.eq('ECO_NAME', 'South Western Ghats montane rain forests'),
  ee.Filter.eq('ECO_NAME', 'North Western Ghats montane rain forests')
));
```

What this does: Filters to keep ONLY the 4 ecoregions that make up the **Western Ghats** mountain range in India (runs from Gujarat to Kerala, ~1,600 km long).

Why 4 names? The Western Ghats has different forest types at different latitudes and elevations:

- North vs South (latitude)
- Moist deciduous vs Montane rain (elevation)

```
var studyArea = westernGhats.geometry();
```

What this does: Extracts just the **boundary shape** (polygon). Every satellite query below will be clipped to this shape — we only care about pixels INSIDE the Western Ghats.

```
Map.centerObject(westernGhats, 7);
Map.addLayer(westernGhats, {color: '228B22'}, 'Western Ghats Boundary');
```

What this does: Centers the map on the Western Ghats at zoom level 7, and draws the boundary in forest-green color. This is just for visual reference when you run the script.

2.2 Temporal Blocks (Lines 63–68)

```
var blocks = [
  {name: 'Block1_2003_2008', start: '2003-01-01', end: '2008-12-31', role: 'TRAIN'},
  {name: 'Block2_2009_2014', start: '2009-01-01', end: '2014-12-31', role: 'TRAIN'},
  {name: 'Block3_2015_2020', start: '2015-01-01', end: '2020-12-31', role: 'TRAIN'},
  {name: 'Block4_2021_2025', start: '2021-01-01', end: '2025-12-31', role: 'TEST'}
];
```

What this does: Divides 22 years of satellite data into 4 chunks called "blocks." Each block is a JavaScript object with:

- `name` : a label for the output file
- `start` / `end` : date range to query
- `role` : whether this block is for training the ML model or testing it

Why these specific years?

- Each block is ~5–6 years = one full **El Niño/La Niña cycle** (ENSO). This ensures each block has both dry and wet periods.
- Blocks 1–3 (2003–2020) = **TRAIN** — the model learns fire patterns from these 18 years.
- Block 4 (2021–2025) = **TEST** — the model is tested on this unseen future data.
- **Block 4 is the one whose predicted points you see on the map.**

2.3 Static Features — Topography and Population (Lines 76–100)

These features are computed **once** because they don't change over time.

Elevation, Slope, Aspect (from SRTM satellite)

```
var dem = ee.Image("USGS/SRTMGL1_003");
```

What is SRTM? The Space Shuttle Radar Topography Mission. In year 2000, the Space Shuttle mapped Earth's elevation at 30-meter resolution. This gives us a Digital Elevation Model (DEM) — a grid where every pixel contains the ground height in meters.

```
var elevation = dem.select('elevation').rename('elevation');
```

What this does: Selects the elevation band from the DEM image. Every pixel now has a value like "850" meaning 850 meters above sea level.

```
var terrainAll = ee.Terrain.products(dem);
var slope  = terrainAll.select('slope').rename('slope_deg');
var aspect = terrainAll.select('aspect').rename('aspect_deg');
```

What this does: `ee.Terrain.products()` is a GEE built-in function that takes the DEM and computes:

- **Slope**: How steep is the terrain at each pixel? Measured in degrees (0° = flat, 90° = vertical cliff).
  - 🔥 Why it matters: Fire spreads **2× faster for every 10° increase in slope** because hot air rises uphill, preheating the unburned fuel above.
- **Aspect**: Which direction does the slope face? (0°=North, 90°=East, 180°=South, 270°=West).
  - 🔥 Why it matters: South-facing slopes in India receive more direct sunlight → drier soil → more fire-prone.

TWI — Topographic Wetness Index

```
var slopeRad = slope.multiply(Math.PI).divide(180);  // degrees → radians
var twi = slopeRad.tan().max(0.001)   // tan(slope), min 0.001 to avoid ÷ by 0
  .pow(-1)                             // 1 / tan(slope)
  .log()                               // ln(1/tan(slope))
  .rename('TWI');
```

Step by step:

1. Convert slope from degrees to radians (computers need radians for trig).
2. Calculate tan(slope) — how steep.
3. `.max(0.001)` — if slope ≈ 0° (flat land), tan = 0, so 1/0 = infinity. This prevents that.
4. Take the reciprocal: 1/tan(slope).
5. Take the natural log.

Result: High TWI = flat area where water accumulates (wet) = less fire risk. Low TWI = steep area where water drains away (dry) = more fire risk.

Population Density

```
var population = ee.ImageCollection("WorldPop/GP/100m/pop")
  .filter(ee.Filter.eq('country', 'IND'))    // India only
  .filter(ee.Filter.eq('year', 2020))         // 2020 snapshot
  .mosaic()                                    // merge tiles into one image
  .rename('population_density');
```

What this does: WorldPop provides gridded population maps. Each pixel = number of people living in that 100m × 100m area. More people = more accidental fires (campfires, cigarette butts, deliberate burning for agriculture).

2.4 The `processBlock()` Function (Lines 107–357)

This is the heart of Script 1. It takes one time block (e.g., 2003–2008) and:

1. Queries multiple satellite datasets for that date range.
2. Computes statistics per pixel.
3. Combines everything into one image with 36 bands.
4. Samples 10,000 points.
5. Returns them as a table with coordinates.

Let me go through each satellite dataset:

A. Active Fire Detection — MOD14A1

```
var mod14 = ee.ImageCollection("MODIS/061/MOD14A1")
  .filterDate(start, end)         // Only images between block start/end dates
  .filterBounds(studyArea);       // Only images covering Western Ghats
```

What is MODIS? Two NASA satellites (Terra and Aqua) that orbit Earth every day. They carry a sensor called MODIS that takes images in 36 different wavelengths, including **thermal infrared** (heat). MOD14A1 is the fire product from Terra.

What is `ImageCollection`? A stack of images over time. From 2003 to 2008, there's one image per day = ~2,190 images. GEE doesn't download them — it processes them on the cloud.

```
var fireImages = mod14.select('FireMask').map(function(img) {
  return img.gte(7).rename('fire').copyProperties(img, ['system:time_start']);
});
```

Step by step:

1. `.select('FireMask')` — Each MOD14A1 image has a band called FireMask. Each pixel value means:
   - 0–6 = no fire (water, cloud, land, unknown, etc.)
   - **7 = low confidence fire**
   - **8 = nominal confidence fire**
   - **9 = high confidence fire**
2. `.map(function(img) {...})` — Apply a function to EVERY image in the collection (all ~2,190 days).
3. `img.gte(7)` — Greater-than-or-equal to 7. Converts to binary: **1 = fire detected, 0 = no fire**.
4. `.rename('fire')` — Give it a clean name.
5. `.copyProperties(...)` — Keep the date stamp from the original image.

Result: A collection of ~2,190 daily binary images — 1 where fire was detected that day, 0 elsewhere.

```
var fireOccurrence = fireImages.max().unmask(0).rename('fire_occurrence');
```

- `.max()` over all ~2,190 daily images = pixel is 1 if fire was detected on **ANY** day in the block, 0 otherwise.
- `.unmask(0)` = if a pixel has no data (masked), set it to 0.
- **This is the TARGET VARIABLE** — what the ML model will try to predict.

> ⚠️ ***CRITICAL**: `fire_occurrence` is **not random**. It comes from real satellite thermal detections. When MODIS flies over a pixel and its thermal sensor detects anomalous heat (much hotter than surroundings), it flags it as fire.*

```
var totalFireCount = fireImages.sum().unmask(0).rename('total_fire_count');
```

- `.sum()` = count how many days fire was detected at this pixel across the entire block.
- A pixel with `totalFireCount = 45` had fire detected on 45 different days (persistent fire zone).

```
var frpMean = mod14.select('MaxFRP').mean().unmask(0).rename('FRP_mean_MW');
var frpMax  = mod14.select('MaxFRP').max().unmask(0).rename('FRP_max_MW');
```

- FRP = **Fire Radiative Power** in megawatts. Measures how **intense** the fire is.
- A small grass fire: ~10 MW. A massive crown fire: ~1000+ MW.
- We take the mean (average intensity) and max (worst intensity) across the block.

B. Burned Area — MCD64A1

```
var mcd64 = ee.ImageCollection("MODIS/061/MCD64A1")
  .filterDate(start, end).filterBounds(studyArea);

var burnedImages = mcd64.select('BurnDate').map(function(img) {
  return img.gt(0).rename('burned');  // BurnDate > 0 = pixel burned this month
});
```

What's different from MOD14A1? MOD14A1 detects **active flames** (needs to see a hot pixel during satellite overpass). MCD64A1 detects **burn scars** — changes in surface reflectance after a fire. It's a monthly product at 500m resolution.

`BurnDate` = day-of-year when the pixel burned (e.g., 45 = February 14). If > 0, it burned; if 0, it didn't.

```
var burnedBinary    = burnedImages.max().unmask(0).rename('burned_area_binary');
var burnMonthCount  = burnedImages.sum().unmask(0).rename('burn_month_count');
```

- `burned_area_binary` = did this pixel burn at all during the block? (0/1)
- `burn_month_count` = how many months did it burn? (fire recurrence measure)

C. FIRMS Brightness & Confidence

```
var firms = ee.ImageCollection("FIRMS")
  .filterDate(start, end).filterBounds(studyArea);

var firmsT21  = firms.select('T21').mean().unmask(0).rename('FIRMS_brightness_K');
var firmsConf =
firms.select('confidence').mean().unmask(0).rename('FIRMS_confidence');
```

- FIRMS = Fire Information for Resource Management System (NASA's near-real-time fire monitoring).
- `T21` = brightness temperature from the 21-micrometer thermal channel, in Kelvin. Higher = hotter fire.
- `confidence` = how confident the algorithm is that this is truly fire (0–100%).

D. Climate/Weather — ERA5-Land

```
var era5 = ee.ImageCollection("ECMWF/ERA5_LAND/MONTHLY_AGGR")
  .filterDate(start, end).filterBounds(studyArea);
```

ERA5-Land is **not a satellite**. It's a **weather reanalysis** product from ECMWF (European weather center). They take global weather station data + satellite data + physics models and create a gridded best-estimate of the atmosphere. Monthly aggregates at ~9 km resolution.

Temperature (Kelvin → Celsius)

```
var tempMean = era5.select('temperature_2m').mean()
  .subtract(273.15)              // Kelvin → Celsius: 300K - 273.15 = 26.85°C
```

```
  .rename('temp_mean_C');
```

- `temperature_2m` = air temperature 2 meters above ground (standard meteorological measurement height).
- `.mean()` = average across all months in the block.
- Also compute `.max()` → hottest month and `.min()` → coldest month.

Precipitation (meters → millimeters)

```
var precipMean  = era5.select('total_precipitation_sum').mean()
  .multiply(1000)                // 0.05 meters = 50 mm
  .rename('precip_mean_mm');
var precipTotal = era5.select('total_precipitation_sum').sum()
  .multiply(1000)
  .rename('precip_total_mm');
```

ERA5 stores precipitation in meters per month. 0.1 meters = 100 mm. Multiply by 1000 to get millimeters (more intuitive).

Wind Speed and Direction

```
var uWind = era5.select('u_component_of_wind_10m').mean();  // East-West wind
var vWind = era5.select('v_component_of_wind_10m').mean();  // North-South wind
var windSpeed = uWind.pow(2).add(vWind.pow(2)).sqrt();      // Pythagorean theorem
```

Wind is measured as two components: u (eastward) and v (northward). The total speed = $\sqrt{(u^2 + v^2)}$. Like if you walk 3 km east and 4 km north, your actual distance from start = $\sqrt{(9+16)}$ = 5 km.

```
var windDir = vWind.atan2(uWind)     // angle in radians
  .multiply(180).divide(Math.PI)     // radians → degrees
  .add(180)                           // adjust to meteorological convention (0°=N)
  .rename('wind_direction_deg');
```

Wind direction in degrees: 0°=North, 90°=East, 180°=South, 270°=West. Important because fire spreads **downwind**.

Relative Humidity (Magnus Formula)

```
// Saturation vapor pressure (max water air CAN hold at temperature T)
var satVP =
airTemp.multiply(17.27).divide(airTemp.add(237.3)).exp().multiply(0.6108);
// Actual vapor pressure (water air currently holds, via dewpoint temperature)
var actVP =
dewTemp.multiply(17.27).divide(dewTemp.add(237.3)).exp().multiply(0.6108);
// RH = actual / max × 100%
var relHumidity = actVP.divide(satVP).multiply(100).min(100).max(0);
```

Analogy: Imagine a sponge (air). At 30°C it can hold 100 ml of water (satVP). Currently it holds 60 ml (actVP). RH = 60/100 × 100 = 60%. Low RH → dry air → fire dries out vegetation faster.

VPD — Vapor Pressure Deficit

```
var vpd = satVP.subtract(actVP).max(0).rename('VPD_kPa');
```

VPD = satVP − actVP = how much MORE water the air WANTS to suck from plants. High VPD = air is aggressively drying the vegetation = **one of the most important fire weather variables**.

Dry Months Count

```
var dryMonths = era5.select('total_precipitation_sum').map(function(img) {
  return img.multiply(1000).lt(20).rename('dry');   // < 20mm = dry month
}).sum().rename('dry_months_count');
```

Counts how many months in the block had rainfall below 20mm. Standard meteorological "dry month" threshold. More dry months → longer fire season.

E. Vegetation — NDVI, EVI, LAI, FPAR, LFMC

```
var mod13 = ee.ImageCollection("MODIS/061/MOD13A2")
  .filterDate(start, end).filterBounds(studyArea);

var ndviMean = mod13.select('NDVI').mean().multiply(0.0001).rename('NDVI_mean');
```

What is NDVI? Normalized Difference Vegetation Index. Plants absorb red light (for photosynthesis) and reflect near-infrared (NIR) light. NDVI = (NIR − Red) / (NIR + Red).

- **NDVI ≈ 0.8**: Dense green forest (lots of chlorophyll).
- **NDVI ≈ 0.2**: Bare soil or very sparse vegetation.
- **NDVI ≈ 0**: Water or rock.
- **Scale factor × 0.0001**: MODIS stores as integers (8000 = NDVI 0.8) for space efficiency.

We compute:

- `NDVI_mean` — average greenness across the block
- `NDVI_min` — driest/most stressed state (peak dry season)
- `NDVI_max` — greenest state (peak monsoon)
- `NDVI_seasonality` (std dev) — how much NDVI fluctuates. Deciduous forests have high seasonality.

LFMC Proxy (Live Fuel Moisture):

```
var lfmc = eviMean.divide(
  ee.Image(1).subtract(ndviMean).max(0.01)
).rename('LFMC_proxy');
```

LFMC = EVI / (1 − NDVI). When vegetation is green and moist (high EVI, high NDVI), LFMC is high (wet fuel, hard to burn). When vegetation is dry (low EVI, low NDVI), LFMC drops (dry fuel, easy to burn).

F. Cross-Modal Features — Combining Different Data Types

```
// Steep slope + dry vegetation = fire spreads fast uphill
var terrainVeg =
slope.multiply(ee.Image(1).subtract(ndviMean)).rename('terrain_x_veg');

// Hot temperature + dry vegetation = high ignition risk
var climateFuel =
tempMean.multiply(ee.Image(1).subtract(ndviMean)).rename('climate_x_fuel');

// Hot + windy + dry = extreme fire weather
var fwiProxy =
tempMean.multiply(windSpeed).divide(precipMean.add(1)).rename('FWI_proxy');

// Dry atmosphere + thin canopy = vulnerable
var drynessVeg = vpd.multiply(laiMean.max(0.1).pow(-1)).rename('dryness_x_fuel');
```

These multiply features from different categories to capture **compound risk** — conditions where multiple danger factors align.

2.5 Combining All 36 Bands into One Image (Lines 292–332)

```
var allBands = fireOccurrence
  .addBands(totalFireCount)
  .addBands(frpMean)
  // ... 30+ more .addBands() calls ...
  .addBands(landCover);

allBands = allBands.clip(studyArea);   // Cut to Western Ghats boundary
```

What this does: Creates a single **multi-band image** — like a Photoshop file with 36 layers, where each layer has different information at each pixel location.

2.6 Stratified Sampling — Creating the Dataset Points (Lines 337–356)

> ⚠️ ***THIS IS WHERE THE DATA POINTS COME FROM.*** *Read carefully.*

```
var sampled = allBands.stratifiedSample({
  numPoints: 5000,
  classBand: 'fire_occurrence',
  region: studyArea,
  scale: 1000,
  seed: 42,
  geometries: true
});
```

Let me explain each parameter:

- **`numPoints: 5000`** — Sample 5,000 points **per class**. Since `fire_occurrence` is binary (0 or 1), this means:
  - 5,000 points where `fire_occurrence = 1` (pixel had fire)
  - 5,000 points where `fire_occurrence = 0` (pixel had no fire)
  - Total = **10,000 points per block**
- **`classBand: 'fire_occurrence'`** — This is the "stratification key." It tells GEE to ensure equal representation of both classes (fire vs. no-fire).
- **`region: studyArea`** — Only sample from within the Western Ghats boundary.
- **`scale: 1000`** — Sample at **1,000 meters (1 km) resolution**. This means the smallest "pixel" is 1km × 1km. This matches MODIS fire detection resolution.
- **`seed: 42`** — Random seed for reproducibility. Running the same code again will produce the exact same 10,000 points.
- **`geometries: true`** — **IMPORTANT**: Keep the geographic coordinates (latitude, longitude) of each sampled point. Without this, we'd get feature values but lose WHERE they are on the map.

🎯 ARE THE POINTS RANDOM?

Answer: They are RANDOMLY LOCATED but DELIBERATELY BALANCED.

- GEE randomly selects 5,000 locations where fire occurred and 5,000 where it didn't.
- The **locations** are random within the Western Ghats (GEE picks them).
- But the **class balance** is forced 50:50 (not random — deliberate).
- The `seed: 42` means the same random locations every time you run it.

The locations are NOT arbitrary — each point is a real 1km × 1km pixel on the ground. All 36 band values at that specific pixel location are extracted and stored.

2.7 Adding Coordinates and Exporting (Lines 346–377)

```
sampled = sampled.map(function(f) {
  var coords = f.geometry().coordinates();
  return f
    .set('block', block.name)           // e.g., "Block4_2021_2025"
    .set('block_role', block.role)       // "TRAIN" or "TEST"
    .set('longitude', coords.get(0))     // e.g., 75.193481
    .set('latitude', coords.get(1));     // e.g., 13.856513
});
```

What this does: For each of the 10,000 sampled points, it:

1. Gets the point's geographic coordinates from `.geometry()`.
2. Adds them as explicit `longitude` and `latitude` columns.
3. Also adds the block name and role as columns.

```
Export.table.toDrive({
  collection: data,
```

```
  description: 'WG_AGFE_' + block.name,       // e.g., "WG_AGFE_Block4_2021_2025"
  fileNamePrefix: 'WG_AGFE_' + block.name,
  folder: 'AGFE_Fire_WesternGhats',
  fileFormat: 'CSV'
});
```

What this does: Saves the 10,000-row table as a **CSV file** on your Google Drive.

The CSV looks like this:

fire_occurrence	total_fire_count	FRP_mean_MW	...	NDVI_mean	elevation
1	23	45.2	...	0.52	850
0	0	0.0	...	0.78	420

4 blocks × 10,000 rows = 4 CSV files, ~40,000 total rows.

---

3. THE JOURNEY: GEE → CSV → Python → Predicted Points

Step 1: Download CSVs

The 4 CSV files are downloaded from Google Drive to your computer.

Step 2: Python Notebook Loads Them

```
df_all = pd.concat([block1_csv, block2_csv, block3_csv, block4_csv])
# Result: one big DataFrame with ~40,000 rows
```

Step 3: Features Are Processed

- Spatial thinning (remove too-close points)
- Multicollinearity removal (VIF)
- Historical fire features added (from prior blocks)

Step 4: Train/Test Split

```
train = blocks 1, 2, 3 (2003-2020)  →  ~30,000 rows
test  = block 4 (2021-2025)          →  ~10,000 rows, but after thinning → ~4,248
rows
```

Step 5: Train ML Model (e.g., Random Forest)

```
rf_model = RandomForestClassifier(...)
rf_model.fit(X_train, y_train)   # learn fire patterns from 2003-2020
```

Step 6: Predict on Test Data

```
y_pred = rf_model.predict(X_test)  # predict fire for each Block 4 pixel
```

For each of the ~4,248 test pixels, the model outputs:

- `pred_RF = 1` → "I think this pixel will have fire"
- `pred_RF = 0` → "I think this pixel will NOT have fire"

Step 7: Save Results with Coordinates

The test data **already has latitude and longitude** from the original GEE CSV export. So the result is:

```
results_df = pd.DataFrame({
    'latitude':  test_latitudes,      # from GEE CSV
    'longitude': test_longitudes,     # from GEE CSV
    'y_true':    y_test,              # actual fire label from GEE
    'pred_RF':   rf_predictions,      # model's prediction
    'proba_RF':  rf_probabilities     # model's confidence
})
results_df.to_csv('test_results_with_coords.csv')
```

Step 8: Extract Coordinates for GEE Visualization

```
# Ground Truth fire locations
gt_fire = results_df[results_df['y_true'] == 1]
gt_coords = gt_fire[['longitude', 'latitude']].values.tolist()
# → [[75.193481, 13.856513], [73.792109, 18.015713], ...]  (2,262 points)

# RF Predicted fire locations
rf_fire = results_df[results_df['pred_RF'] == 1]
rf_coords = rf_fire[['longitude', 'latitude']].values.tolist()
# → [[75.579756, 14.099058], [74.968902, 15.239919], ...]  (1,986 points)
```

These coordinate arrays are then **copy-pasted** into the visualization GEE script.

📌 THE KEY INSIGHT: REVERSE MAPPING

```
FORWARD:  GEE → satellite data at pixel → CSV with lat/lon + features
REVERSE:  CSV results with lat/lon → hardcode into GEE script → plot on map
```

The "reverse mapping" is simply using the same latitude/longitude values that GEE originally created. The coordinates travel with the data through the entire pipeline:

1. **GEE creates them** during `stratifiedSample()` with `geometries: true`.
2. **GEE exports them** as `latitude` and `longitude` CSV columns.
3. **Python preserves them** throughout training/testing.
4. **Python saves them** in the results CSV.
5. **We hardcode them** into the visualization script as JavaScript arrays.
6. **GEE Script 2 converts them back** to map points with `ee.Geometry.Point()`.

Nothing is re-generated or approximated. The exact same coordinates come back full circle.

---

4. SCRIPT 2: Visualization — `GEE_RF_FirePrediction_Visualization.js`

4.1 Embedded Coordinates (Lines 26–29)

```
var gtFireCoords = [[75.193481,13.856513],[73.792109,18.015713],...];  // 2,262
pairs
var rfFireCoords = [[75.579756,14.099058],[74.968902,15.239919],...];  // 1,986
pairs
```

What this is: Two massive JavaScript arrays, each containing `[longitude, latitude]` pairs. These are the **exact coordinates from the Python results CSV** (Step 8 above).

Why hardcoded? GEE can't read CSV files from your computer or Google Drive programmatically. The only way to get external data into a GEE script is to either:

1. Upload it as a GEE Asset (complex process), or
2. **Paste the data directly into the code** (simpler, what we do here).

`gtFireCoords` = Ground Truth fire locations where `y_true == 1` (MODIS actually detected fire there during 2021–2025). There are **2,262** such points.

`rfFireCoords` = Random Forest predicted fire locations where `pred_RF == 1` (the ML model predicted fire there). There are **1,986** such points.

4.2 Study Area (Lines 35–44)

```
var ecoregions = ee.FeatureCollection("RESOLVE/ECOREGIONS/2017");
var westernGhats = ecoregions.filter(ee.Filter.or(
  ee.Filter.eq('ECO_NAME', 'South Western Ghats moist deciduous forests'),
  // ... same 4 ecoregions ...
));
var studyArea = westernGhats.geometry();
Map.centerObject(westernGhats, 7);
```

Identical to Script 1. Re-defines the same Western Ghats boundary to clip background layers and provide the green boundary overlay.

4.3 Converting Coordinate Arrays to Map Points (Lines 52–60)

```
function coordsToFC(coordArray) {
  var features = coordArray.map(function(c) {
    return ee.Feature(ee.Geometry.Point(c[0], c[1]));
  });
  return ee.FeatureCollection(features);
}
```

What this function does, step by step:

1. Takes an array like `[[75.19, 13.85], [73.79, 18.01], ...]`.
2. `.map()` loops over each `[lon, lat]` pair.
3. `ee.Geometry.Point(c[0], c[1])` — Creates a GEE Point geometry from `longitude` (c[0]) and `latitude` (c[1]).
4. `ee.Feature(...)` — Wraps the point in a Feature object (GEE's basic vector data unit).
5. `ee.FeatureCollection(features)` — Combines all Features into a FeatureCollection (like a shapefile).

```
var gtFC = coordsToFC(gtFireCoords);   // 2,262 blue dots (Ground Truth)
var rfFC = coordsToFC(rfFireCoords);   // 1,986 red dots (RF Predicted)
```

Now `gtFC` and `rfFC` are GEE FeatureCollections — they can be displayed on the map, styled with colors, and used for spatial queries.

4.4 Computing Overlap: True Positives, False Positives, False Negatives (Lines 62–79)

True Positives (Yellow) — RF said fire AND there WAS fire

```
var tpFC = rfFC.map(function(feat) {
  var nearby = gtFC.filterBounds(feat.geometry().buffer(1000));
  return feat.set('is_tp', nearby.size().gt(0));
}).filter(ee.Filter.eq('is_tp', 1));
```

Line by line:

1. `rfFC.map(function(feat) {...})` — Loop over every RF predicted point.
2. `feat.geometry().buffer(1000)` — Create a 1,000-meter (1 km) circle around this RF point.
3. `gtFC.filterBounds(...)` — Find all Ground Truth points that fall inside this circle.
4. `nearby.size().gt(0)` — Is there at least 1 GT point nearby? If yes → True Positive.
5. `.filter(ee.Filter.eq('is_tp', 1))` — Keep only the True Positives.

Why 1 km buffer? Because sampling happened at 1 km MODIS resolution. Two "same location" points might differ by a few hundred meters due to pixel-center alignment. 1 km buffer accounts for this.

False Positives (Orange) — RF said fire BUT there was NO fire

```
var fpFC = rfFC.map(function(feat) {
  var nearby = gtFC.filterBounds(feat.geometry().buffer(1000));
  return feat.set('is_fp', nearby.size().eq(0));   // eq(0) = NO GT fire nearby
}).filter(ee.Filter.eq('is_fp', 1));
```

Same logic, but keeps RF points that have **zero** GT neighbors within 1 km. These are **false alarms**.

False Negatives (Purple) — There WAS fire BUT RF didn't predict it

```
var fnFC = gtFC.map(function(feat) {
  var nearby = rfFC.filterBounds(feat.geometry().buffer(1000));
  return feat.set('is_fn', nearby.size().eq(0));   // NO RF prediction nearby
}).filter(ee.Filter.eq('is_fn', 1));
```

Now we loop over **GT points** and check if any RF point is nearby. GT points with NO RF prediction = **missed fires**.

4.5 Background Map Layers (Lines 86–122)

Western Ghats Boundary (Green)

```
Map.addLayer(westernGhats, {color: '228B22'}, '🌿 Western Ghats Boundary', true);
```

Draws the boundary polygon in forest green. `true` = visible by default.

MODIS Fire Density Heatmap

```
var mod14 = ee.ImageCollection("MODIS/061/MOD14A1")
  .filterDate('2021-01-01', '2025-12-31')
  .filterBounds(studyArea);

var fireCount = mod14.select('FireMask').map(function(img) {
  return img.gte(7);
}).sum().clip(studyArea);

Map.addLayer(fireCount,
  {min: 0, max: 30, palette: ['FFFFFF','FFFF80','FFA500','FF4500','8B0000']},
  '🔥 MODIS Fire Density 2021-2025 (background)',
  true, 0.5);   // visible at 50% opacity
```

This creates a **continuous heatmap** showing total fire detection count per pixel for the entire Block 4 period. White = 0 fires, dark red = 30+ fire days. Shown at 50% opacity as a subtle background.

NDVI Vegetation (Hidden by Default)

```
var ndvi = ee.ImageCollection("MODIS/061/MOD13A2")
  .filterDate('2021-01-01', '2025-12-31')
  .filterBounds(studyArea)
  .select('NDVI').median().multiply(0.0001).clip(studyArea);

Map.addLayer(ndvi,
  {min: 0, max: 0.9, palette:
['CE7E45','DF923D','F1B555','FCD163','99B718','529400','207401']},
  '🌿 NDVI Median 2021-2025 (background)',
  false);   // hidden by default — user can toggle on
```

Brown → yellow → green color ramp. Dense forest = dark green. Useful to see if fire predictions align with dry/sparse vegetation.

4.6 Model Result Layers (Lines 128–167)

```
// LAYER A: Ground Truth (Blue)
Map.addLayer(gtFC, {color: '0000FF'},
  '🔵 Ground Truth Fire (y_true=1) — 2262 pts', true);

// LAYER B: RF Predicted (Red)
Map.addLayer(rfFC, {color: 'FF0000'},
  '🔴 RF Predicted Fire (pred_RF=1) — 1986 pts', true);

// LAYER C: True Positives (Gold/Yellow)
Map.addLayer(tpFC, {color: 'FFD700'},
  '🟡 True Positives (GT=1 & RF=1)', true);

// LAYER D: False Positives (Orange) — hidden by default
Map.addLayer(fpFC, {color: 'FF6600'},
  '🟠 False Positives (RF=1 but GT=0)', false);

// LAYER E: False Negatives (Purple) — hidden by default
Map.addLayer(fnFC, {color: '9400D3'},
  '🟣 False Negatives (GT=1 but RF=0)', false);
```

Each `Map.addLayer()` call adds a layer to the interactive map. The last parameter (`true` / `false`) controls default visibility. Users can toggle any layer using the **Layers panel** in the top-right corner of the GEE map.

4.7 Legend Panel (Lines 173–232)

```
var legend = ui.Panel({
  style: {
    position: 'bottom-left',
    padding: '8px 12px',
    backgroundColor: 'rgba(255,255,255,0.9)'
```

```
  }
});
```

Creates a floating panel widget positioned at the bottom-left of the map. Semi-transparent white background.

```
var items = [
  {color: '0000FF', label: 'Ground Truth Fire  (2262 pts)'},
  {color: 'FF0000', label: 'RF Predicted Fire  (1986 pts)'},
  {color: 'FFD700', label: 'True Positives  (GT ∩ RF)'},
  {color: 'FF6600', label: 'False Positives  (RF only)'},
  {color: '9400D3', label: 'False Negatives  (GT only)'}
];

items.forEach(function(item) {
  var row = ui.Panel({...});       // horizontal row
  var dot = ui.Label({             // colored square
    style: {
      backgroundColor: '#' + item.color,
      padding: '6px',              // 6px padding = small colored box
      border: '1px solid #333'
    }
  });
  var lbl = ui.Label({value: item.label});   // text label
  row.add(dot); row.add(lbl); legend.add(row);
});

Map.add(legend);
```

For each legend item, creates a horizontal row with a colored square + text label. The colored square is a `ui.Label` styled with `backgroundColor` (no image, just CSS trick). Finally, adds the legend panel to the map.

---

5. FAQ — Your Specific Questions Answered

Q1: "Are the predicted data points random?"

NO. The predicted points are NOT randomly placed on the map. Here's the exact chain:

Step	What Happens	Who Decides the Location?
1	GEE samples 10,000 real satellite pixels per block	GEE's `stratifiedSample()` (semi-random, but real pixel locations)
2	These exact locations are saved as lat/lon in the CSV	GEE embedded coordinates
3	Python trains ML model on features at these locations	Features from the pixel locations

4	Python predicts fire/no-fire for Block 4 test locations	The model decides, based on environmental features
5	Points where model says fire=1 are extracted with lat/lon	**Same coordinates from Step 1**
6	These lat/lon pairs are pasted into the GEE viz script	Exact same numbers, copied
7	GEE places dots at those exact locations on the map	Same pixel centers from Step 1

The randomness happens ONLY in Step 1 (which pixels GEE chooses to sample). After that, the same coordinates travel through the entire pipeline unchanged.

The **ML model doesn't invent new locations** — it only decides whether each existing Block 4 location is fire=1 or fire=0.

Q2: "How is the CSV mapped to GEE?"

Forward mapping (Script 1 → CSV):

```
GEE pixel at (75.193, 13.856) → Extract 36 feature values → Write row to CSV with lat/lon
```

Reverse mapping (CSV → Script 2):

```
CSV row: lat=13.856, lon=75.193, pred_RF=1 → Hardcode [75.193, 13.856] into JS array → GEE: ee.Geometry.Point(75.193, 13.856) → place red dot on map at that location
```

It's the SAME coordinate used in BOTH scripts. No transformation, no approximation.

Q3: "Why are there 2,262 GT fire points but only 1,986 RF predicted fire points?"

Because the RF model is **conservative** — it predicts fire for fewer pixels than actually had fire.

- **2,262 GT fires**: MODIS satellite detected thermal anomalies at 2,262 test locations during 2021–2025. This is observational truth.
- **1,986 RF predictions**: The Random Forest model, based on environmental features (temperature, NDVI, wind, slope, fire history), predicted that 1,986 locations would have fire.
- The model **missed 276 fire locations** (false negatives where GT=1 but RF=0).
- The model also **incorrectly predicted some non-fire locations as fire** (false positives where RF=1 but GT=0).

Q4: "Where exactly are the 1,986 predicted fire points?"

They are scattered across the Western Ghats, concentrated in **known fire-prone zones**:

- **Northern Western Ghats**: Maharashtra/Goa border forests (latitude ~16–19°N)
- **Central Western Ghats**: Karnataka deciduous forests (latitude ~13–16°N)

- **Southern Western Ghats**: Kerala-Tamil Nadu highlands (latitude ~8–12°N)

The distribution closely follows the MODIS fire density heatmap (the red/orange background layer), because the model learned that high-temperature, low-rainfall, low-NDVI, steep-slope, historically-burned areas are fire-prone.

Q5: "What proves that the model is working, not just guessing?"

1. **True Positives (Yellow dots)**: When you toggle the yellow layer, you see that MOST blue (GT) and red (RF) dots overlap. The model correctly identifies fire locations.
2. **Spatial coherence**: Predicted fire points cluster in the same corridors as observed fires — not scattered randomly across the map.
3. **Quantitative metrics** (from the notebook): Accuracy %, F1-score, AUC-ROC — all well above random chance (50%).